MODULE 02 · DOMAIN 2 - TERRAFORM FUNDAMENTALS

Architecture: Core, Providers, the Registry

How Terraform reaches an API it knows nothing about, and how you tell it which plugin to use.

DOMAIN 2 of 8 COURSE WEIGHT 12% MODULE 2 of 27

admin@sthithapragna.com

Terraform core knows nothing about any cloud. **Every** piece of platform knowledge lives in a plugin.

Two halves, released separately

- **Terraform core.** Parses configuration, builds the dependency graph, manages state, decides what must change.
- **Providers.** Plugins that translate those decisions into real API calls against a specific platform.
- Providers are separate programs, written in Go, that core launches and talks to over a documented plugin protocol.
- **They version independently.** Terraform 1.12 and AWS provider 5.90 are two different version numbers with two different meanings.
- A provider is **not** part of the Terraform binary and is not downloaded until you run **terraform init**.

> ON THE EXAM
>
> Independent versioning is why required_version and required_providers are two separate settings. They constrain different things.

What happens when you declare a resource

01
Declare
You write a resource with a type such as aws_instance.

02
Resolve
Core reads the first word of the type to find the provider's local name.

03
Launch
Core starts the installed plugin and hands it the configuration.

04
Translate
The provider turns create, read, update or delete into real API calls.

05
Return
The provider reports back what exists, and core records it in state.

ON THE EXAM

The first word of a resource type is not decoration. It is how Terraform chooses which plugin handles that resource.

What a provider is responsible for

- **Defining resource types and data sources.** Everything you can manage comes from some provider.
- **Authenticating** to the platform, and knowing its endpoints and regions.
- Mapping Terraform's lifecycle onto that platform's API, including what forces a replacement rather than an update.
- **Over three thousand providers** exist: clouds, on-premises systems, SaaS, DNS, monitoring, databases, version control.
- A provider does **not** hold state, plan changes, or order operations. Those are core's job.

> ON THE EXAM
>
> If a behaviour is platform-specific, it is the provider's. If it is about graphs, plans or state, it is core's.

The three registry tiers

TIER	WHO MAINTAINS IT
Official	HashiCorp. Published in the hashicorp namespace
Partner	A third party in HashiCorp's Technology Partner Program, with a verified publisher
Community	An individual or organisation from the community. No formal verification

ON THE EXAM

The tier tells you who supports it, not how it installs. All three install identically with terraform init.

A source address has three parts

- **[HOSTNAME/]NAMESPACE/TYPE**. For example, registry.terraform.io/hashicorp/aws.
- **Hostname** is optional and defaults to registry.terraform.io, the public registry.
- **Namespace** is the publishing organisation. **Type** is the platform, and must be unique in that namespace.
- The address is the provider's **global identifier**. Terraform normalises it and uses it internally to tell providers apart.
- Republish the same provider under a different namespace and Terraform treats it as an **entirely different provider**.

> ON THE EXAM
>
> Source addresses are case-insensitive, but write types in lowercase. hashicorp/aws and registry.terraform.io/hashicorp/aws are the same provider.

Source address or local name?

SOURCE ADDRESS

Global, and used once

- Identifies the provider worldwide
- Appears only in required_providers
- Written as namespace and type
- Same for everyone, everywhere

LOCAL NAME

Per-module, and used everywhere else

- Your name for it inside this module
- Must be unique within the module
- Used by provider blocks and resource prefixes
- You may choose it, but rarely should

THE TIEBREAKER

The local name is the key in required_providers. The source address is its value. Everything after that refers to the local name.

required_providers: declare before you configure

- Nested inside the top-level **terraform** block. Every module should declare the providers it requires.
- Each entry takes a **source** and, strongly recommended, a **version** constraint.
- **Omit source and Terraform implies registry.terraform.io/hashicorp/ plus your local name.** A backwards-compatibility rule from 0.13.
- That implication is why a community provider silently resolves to a HashiCorp one that may not exist.
- **terraform init** reads this block, downloads the plugins, and records exact versions in the lock file.

> ON THE EXAM
>
> Always write source explicitly. The implied hashicorp namespace is a transitional aid, not a convenience.

required_providers or a provider block?

REQUIRED_PROVIDERS

Which plugin, and which version

- Inside the terraform block
- Names the source address
- Constrains the version
- Read by terraform init to install

PROVIDER BLOCK

How that plugin should behave

- Top-level block in your configuration
- Region, endpoints, credentials, defaults
- Optional if the provider needs no settings
- Read at plan and apply time

THE TIEBREAKER

Declaration installs. Configuration parameterises. A provider block alone will not install anything.

Configuring a provider, and authenticating it

- The **provider** block is labelled with the **local name**, not the source address.
- Arguments are provider-specific. Region, endpoint and profile are typical.
- **Never hardcode credentials in a provider block.** The configuration is version-controlled; the credentials would be too.
- Prefer **environment variables**, the platform's own credential chain, or a dynamically issued short-lived credential.
- HCP Terraform's answer is **dynamic credentials**: short-lived, issued per run, never stored. Module twenty-one.

> ON THE EXAM
>
> Same rule as module one's secrets scenario, one level lower. Configuration in version control, credentials never.

Aliases: several configurations of one provider

- A provider block with an **alias** argument creates an **additional** configuration alongside the default.
- The block without an alias is the **default**, used by any resource that does not say otherwise.
- **This is the answer to two regions, two accounts, or two subscriptions** in one configuration.
- A resource selects one with the **provider** meta-argument, written as local name dot alias.
- Aliases do not install anything extra. It is **one plugin**, configured more than once.

> ON THE EXAM
>
> Objective 2c is about writing configuration with multiple providers. Aliases are how you use the same provider twice.

Which configuration does a resource use?

THE SITUATION	WHAT TERRAFORM DOES
`No provider meta-argument`	The default configuration for the type prefix
`provider = aws.tokyo`	The aliased configuration named
`Two providers, same type name`	You must set provider on every resource
`Child module needs an alias`	Pass it with the providers meta-argument
`Child module defines its own`	Discouraged. Pass configuration in instead

ON THE EXAM

Modules inherit default configurations only. Aliased ones must be passed in.

When two providers want the same name

- Local names must be unique per module. Two providers of type **http** from different namespaces will collide.
- HashiCorp's recommendation: **compound local names** joining namespace and type, such as hashicorp-http and mycorp-http.
- Terraform can then no longer infer the provider from the resource type prefix.
- So every affected resource and data source must carry an explicit **provider** meta-argument.
- More typing, and deliberately so. Ambiguity here is far more expensive than verbosity.

> ON THE EXAM
>
> Renaming breaks the type-prefix shortcut. That is the cost, and it is why preferred local names are the default advice.

Two providers that break the pattern

- **The built-in provider.** Its source address is terraform.io/builtin/terraform and it powers terraform_remote_state.
- It is **inside the Terraform binary**, so you never declare it and terraform init never downloads it.
- **hashicorp/terraform is a different, obsolete thing.** It should never appear in required_providers.
- **In-house providers** need not be published. Use a hostname on a domain you control, such as terraform.example.com.
- Distribute them through a **filesystem mirror** or a private registry. The address needs a hostname; the hostname need not resolve.

> ON THE EXAM
>
> A source address is an identifier first and a download location second. It does not have to point at a real service.

Where the plugins actually land

- **terraform init** downloads providers into the **.terraform** directory inside your working directory.
- **.terraform is disposable and belongs in .gitignore.** It is a working directory, not part of your configuration.
- The **lock file** is the opposite: .terraform.lock.hcl is committed. Module three covers it properly.
- Provider binaries are large. Downloading them per project, per branch, adds up quickly.
- A **plugin cache directory** shares one copy across every configuration on the machine.

> ON THE EXAM
>
> Two files named almost identically, opposite treatment. Ignore the .terraform directory. Commit the .terraform.lock.hcl file.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Two regions or two accounts	Provider aliases, and the provider meta-argument
Wrong provider installed, or not found	source omitted; the hashicorp namespace implied
Proprietary system, never published	In-house provider from a filesystem mirror
Which versions are acceptable	required_providers, not the provider block
Which region the provider uses	The provider block, not required_providers
Child module needs the alias	The providers meta-argument on the module block

ON THE EXAM

Ask whether the requirement is about installing a plugin or configuring one.

> Coriander Media runs its main platform in one cloud region. A new regulatory requirement means customer records for one market must be stored in a second region, in the same cloud account, while the rest of the platform stays where it is.
>
> The team wants both sets of resources described in a single configuration, planned and applied together, so the dependency between them is visible.

How should the team configure this?

A Declare the provider twice in required_providers, once per region

B Add a second provider block with an alias, and set the provider meta-argument on the second region's resources

C Create a second root module and apply it separately

D Use a CLI workspace per region, switching between them before each apply

◆ PAUSE AND DECIDE

How should the team configure this?

A Declare it twice

required_providers holds one entry per local name. It says which plugin to install, and region is not part of that. Duplicate keys are invalid.

B Alias plus meta-argument

Correct. One plugin, two configurations. Both regions appear in one plan, so the dependency between them is real and visible.

C A second root module

Works, and splits the plan in two. The scenario explicitly asks for one plan, which is the sentence that rules this out.

D A workspace per region

CLI workspaces give one configuration several independent states. Only one is active at a time, so the two regions could never appear in one plan.

THE SIGNAL

Planned and applied together, with the dependency visible, is the deciding sentence. Anything that splits the plan fails it.

> An engineer adds a DNS provider published by a vendor called Northwind to a module. She writes a required_providers entry with the local name dns and a version constraint, then runs terraform init.
>
> Terraform reports that it has installed a provider, but every Northwind resource type in her configuration is rejected as unknown. The version installed does not match anything on Northwind's release page.

What is the most likely cause?

A. Community providers cannot be installed automatically by terraform init

B. The source argument was omitted, so Terraform resolved hashicorp/dns instead

C. The dependency lock file pinned an older version and must be deleted

D. The local name must always match the vendor's namespace, not the type

◆ PAUSE AND DECIDE

What is the most likely cause?

A Cannot install community providers

True before Terraform 0.13, and false ever since. Official, partner and community providers all install identically from the registry.

B source omitted, hashicorp implied

Correct. With no source, Terraform implies registry.terraform.io/hashicorp/ plus the local name, so it installed hashicorp/dns.

C Stale lock file

A lock file constrains versions of the provider it already recorded. It cannot cause a different provider to be chosen.

D Local name must match namespace

Invents a rule. Local names are yours to choose and conventionally match the type, never the namespace.

THE SIGNAL

It installed successfully and the resource types are unknown. That combination means the wrong provider, not a broken install.

A bank has written its own Terraform provider for an internal mainframe scheduling system. The provider must be usable by about forty teams across the organisation.

Security has stated that neither the provider binary nor its documentation may be published anywhere outside the bank's own network. The platform team would rather not operate another internal service if it can be avoided.

What should the platform team do?

A Publish it to the public registry under a private namespace

B Give it a source address on a bank-controlled hostname and distribute it via a filesystem mirror

C Omit it from required_providers and have each team place the binary manually

D Rewrite the functionality as a module instead, since providers must be publicly published

◆ PAUSE AND DECIDE

What should the platform team do?

A Public registry, private namespace

The public registry has no private namespaces. A namespace is an organisation name, not an access control, and everything published there is public.

B Own hostname plus filesystem mirror

Correct. The address identifies the provider; it need not resolve. A mirror avoids running a registry, and a real one can be added later at the same address.

C Omit and place manually

Omitting source implies the hashicorp namespace, which is scenario two's failure. Providers must be declared to be used.

D Rewrite as a module

A module composes existing resource types. It cannot invent one, so it cannot reach a system no provider knows about.

THE SIGNAL

A source address is an identifier that happens to describe a download location. The hostname does not have to exist.

> A configuration needs HashiCorp's http provider to fetch a document, and a second http provider published by a monitoring vendor to register a health check. Both providers use http as their preferred local name.
>
> An engineer adds both to required_providers and terraform init fails before downloading anything.

What must the engineer do to make this work?

A. Give each an alias in its provider block, keeping the local name http for both

B. Use distinct compound local names, then set the provider meta-argument on every affected resource

C. Split the two providers into separate child modules so the names never meet

D. Keep only the HashiCorp provider, since duplicate type names are not supported

◆ PAUSE AND DECIDE

What must the engineer do to make this work?

A Alias them both

Aliases distinguish configurations of one provider. These are two different plugins with two different source addresses, so the collision is unresolved.

B Compound names plus meta-argument

Correct, and both halves are required. Unique names let init proceed; the meta-argument replaces the type-prefix inference that renaming destroyed.

C Separate child modules

Local names are per-module, so this genuinely resolves the collision. It also splits related resources apart to satisfy a naming rule, which is the tail wagging the dog.

D Drop one provider

Duplicate type names across namespaces are explicitly supported. This abandons a stated requirement to avoid a solved problem.

THE SIGNAL

Option A is the trap. Aliases separate configurations of one provider; they cannot separate two different providers.

Module 2 recap

- **Core knows nothing platform-specific.** Providers are separate plugins, separately versioned.
- **Source address is global; local name is per-module.** The local name is the key, the source is the value.
- **Omit source and you get the hashicorp namespace.** Always write it explicitly.
- **required_providers installs. The provider block configures.** Different jobs, constantly confused.
- **Aliases give one provider several configurations.** Two regions or two accounts in one plan.
- Modules inherit **default** configurations only. Aliased ones must be passed in.

> ON THE EXAM
>
> Official, Partner and Community describe who supports a provider. They install identically.